

Documentacion Tecnica - Plataforma Comunidad ESAP

Estado levantado directamente desde el codigo fuente del repositorio.

Fecha de actualizacion: **30 de marzo de 2026**

Este documento reemplaza la version anterior (orientada a arquitectura objetivo) por una version **as-is**.

0. Control del documento

Campo	Valor
Proyecto	Plataforma Comunidad ESAP
Tipo	Documentacion tecnica de arquitectura y operacion
Version	2026.03 (as-is)
Fuente	Monorepo frontend + backend + db + docker
Entornos considerados	Local, DEV, QA, PRE, PROD

1. Resumen ejecutivo

La plataforma esta implementada como:

- **Frontend SPA** en React + TypeScript + Vite.
- **Backend de microservicios** en NestJS.
- **API Gateway** con ruteo por servicio y version en URL.
- **PostgreSQL** como base principal (esquemas por dominio).
- **Redis** como soporte de cache/session en servicios que lo usan.
- **Nginx** para servir frontend y hacer proxy a `/services/*`.
- Integraciones externas relevantes: **Microsoft OAuth**, **Microsoft Graph**, **OnlyOffice**, **SMTP**.

1.1 Estado de madurez real

- Servicios con mayor desarrollo funcional: `auth-service`, `academic-registration-service`, `certification-service`, `internal-disciplinary-control-service`, `internal-institutional-control-service`, `legal-management-service`.
- Servicios base/minimos (scaffold): `academic-work-plan-service`, `interoperability-service`, `travel-expenses-service`.

- La arquitectura backend si es de microservicios, pero en frontend todavia existe deuda de consolidacion en la capa API (clientes duplicados y endpoints heredados).

2. Alcance de esta version

Incluye:

- Arquitectura de frontend y backend real.
- Contrato de ruteo del gateway.
- Catalogo de microservicios y dominios funcionales.
- Modelo de datos por esquemas y estrategia de migraciones.
- Arquitectura de despliegue por entornos.
- Integraciones externas activas.
- Brechas tecnicas detectadas con prioridad sugerida.

No incluye:

- Manual funcional por pantalla.
- Inventario campo a campo de cada entidad.
- Catalogo exhaustivo de todos los endpoints CRUD.

3. Arquitectura de alto nivel

Vista general estilo infraestructura cloud (incluye build y despliegue con Docker):

Export PNG del diagrama tecnico: `arquitectura_esap_cloud.png`

4. Estructura del repositorio

```
.
|-- src/                                # Frontend React
|-- backend/
|   |-- api-gateway/
|   |-- auth-service/
|   |-- academic-registration-service/
|   |-- academic-work-plan-service/
|   |-- certification-service/
|   |-- internal-disciplinary-control-service/
|   |-- interoperability-service/
|   |-- internal-institutional-control-service/
|   |-- legal-management-service/
|   |-- notifications-service/
|   |-- travel-expenses-service/
|   |-- audit-service/
|-- db/
```

```
| |-- init/                                # DDL y seed inicial
| |-- migrations/                          # Migraciones incrementales
|-- docker-compose*.yaml
|-- Dockerfile.frontend*
|-- deploy.dev.sh / qa / pre / prod
|-- nginx.conf
```

5. Arquitectura frontend

5.1 Stack real

- React 18 + TypeScript
- Vite 6
- React Router (`BrowserRouter`)
- UI: Radix + utilidades Tailwind
- Estado/caching: combinacion de estado local/context + hooks
- Cliente HTTP: `fetch` /XHR encapsulados en servicios propios

5.2 Entry points

- `src/main.tsx`: monta `BrowserRouter` + `App`.
- `src/App.tsx`: orquestador principal de vistas publicas, portal y backoffice.

5.3 Vistas principales

La SPA combina 3 bloques:

- **Landing publica**
- **Portal transaccional**
- **Backoffice administrativo**

Rutas publicas explicitas en `App.tsx`:

- `/verificar-certificado-graduado`
- `/certificacion-titulos-graduados`
- `/solicitar-certificado-graduado` (redirect)
- `/verificar-certificado/:codigo`
- `/validar/:codigo`
- `/editor-plantillas`
- `/expediente-compartido/:token`

El resto cae en render dinamico de vistas internas.

5.4 Backoffice modular (carga lazy)

`src/components/esap/BackofficeApp.tsx` carga modulos por demanda.

Modulos destacados del backoffice:

- Gestion de usuarios/personas
- Roles/permisos
- Estructura organizacional
- Programas academicos
- Certificados laborales
- Registro academico (graduados/certificados)
- Control interno institucional
- Control disciplinario
- Gestion legal
- Gestion profesoral
- Modulos de comunidad y reportes

5.5 Sesion y autenticacion en frontend

- Persistencia de sesion en `localStorage`.
- Timeout de inactividad de 15 minutos con alerta previa.
- Flujo de login clasifica acceso a `portal` o `backoffice` segun roles/modulo.
- Soporte de login Microsoft via endpoint dedicado.

5.6 Capa API frontend

Componentes centrales:

- `src/config/environment.ts`
- `src/services/api/apiClient.ts`
- `src/services/api/client.ts`

Patrones actuales:

- Modo **gateway**: `/ {service} /api/v{n} / {path} .`
- Modo **directo**: llamada al puerto especifico del microservicio.
- Construccion de URLs por servicio con `buildApiUrl`.

6. Arquitectura backend

6.1 API Gateway

Ubicacion: `backend/api-gateway`

6.1.1 Contrato de ruteo

Patron principal:

- `/ {service} /api/v:version/ *`

Fallback:

- `/ {service} /api/ *` -> redireccion logica a v1

Rutas estaticas proxificadas:

- `/ {service} /uploads/ *`
- `/ {service} /files/ *`
- `/ {service} /disciplinary-autos/ *`
- `/ {service} /disciplinary-oficios/ *`

6.1.2 Seguridad transversal

- Guard JWT global (`JwtAuthGuard`) en gateway.
- Lista de rutas publicas por patron regex:
 - `login/auth` basico
 - validaciones publicas de certificados
 - autoservicios de certificados
 - recursos publicos de archivos
 - expediente compartido disciplinario

6.1.3 Auditoria transversal

- Interceptor global registra operaciones mutables (`POST/PUT/PATCH/DELETE`).
- Envia log asincrono a `audit-service` para no bloquear respuesta.

6.2 Catalogo de microservicios

Servicio	Puerto	Alias de gateway	Estado funcional
<code>auth-service</code>	3001	<code>auth</code>	Maduro
<code>academic-registration-service</code>	3002	<code>registro-academico</code>	Maduro
<code>academic-work-plan-service</code>	3003	<code>pta</code>	Base/scaffold
<code>certification-service</code>	3004	<code>certificados</code> , <code>certificates</code>	Maduro
<code>internal-disciplinary-control-service</code>	3005	<code>control-disciplinario</code>	Maduro
<code>interoperability-service</code>	3006	<code>interoperabilidad</code>	Base/scaffold

Servicio	Puerto	Alias de gateway	Estado funcional
internal-institutional-control-service	3007	control-institucional	Maduro
legal-management-service	3008	legal, legal-management	Maduro
notifications-service	3009	notificaciones	Funcional focalizado
travel-expenses-service	3010	viaticos	Base/scaffold
audit-service	3011	audit	Funcional focalizado

6.3 Volumetria tecnica (codigo)

Conteo aproximado de `src` (TS), controladores, servicios y entidades:

Servicio	Archivos TS	Controllers	Services	Entities
auth-service	44	7	7	12
academic-registration-service	28	3	3	9
certification-service	31	3	5	13
internal-disciplinary-control-service	103	22	25	22
internal-institutional-control-service	196	21	28	46
legal-management-service	133	31	32	45
notifications-service	15	2	2	0
audit-service	14	2	2	1
academic-work-plan-service	9	1	1	0
interoperability-service	9	1	1	0
travel-expenses-service	9	1	1	0

6.4 Dominios funcionales por servicio

6.4.1 `auth-service`

- Login/password
- Login Microsoft
- Verificacion de sesion
- Usuarios, roles, modulos

- Estructura organizacional
- Programas academicos

6.4.2 academic-registration-service

- Gestion de graduados
- Solicitudes y generacion de certificados de grado
- Autoservicio publico y validaciones
- Soporte de descargas/validaciones por QR y codigo

6.4.3 certification-service

- Solicitudes de certificados laborales
- Generacion y verificacion publica de certificados
- Configuracion de plantillas, logos, firmas
- Gestion de prima tecnica

6.4.4 internal-disciplinary-control-service

- Noticias disciplinarias
- Procesos disciplinarios
- Gestion de autos, oficios y actas
- Terminos procesales, reglas y alertas
- Tareas/notas por proceso
- Exportaciones
- Compartir expediente con token publico
- Integracion OnlyOffice

6.4.5 internal-institutional-control-service

- Plan anual (roles/actividades)
- Auditorias y universo auditable
- Hallazgos y planes de mejoramiento
- Informes de ley
- Listas de chequeo
- Kanban/tareas de auditoria
- Notificaciones y evidencias documentales

6.4.6 legal-management-service

- Expedientes, actuaciones, autos, audiencias, terminos
- Riesgos y planes de mejoramiento
- Consultas juridicas
- Requerimientos OC
- Procesos coactivos
- Oficios y plantillas

- Correo juridico con Microsoft Graph
- Clasificación inteligente de correos (modelo bayesiano + heurísticas)

6.4.7 notifications-service

- API de envío de correos:
 - código de validación
 - envío simple
 - envío con adjunto base64

6.4.8 audit-service

- Recepción de logs de peticiones
- Consulta de logs y estadísticas

7. Datos y persistencia

7.1 Base de datos principal

- Motor: PostgreSQL 16 (Docker en todos los entornos).
- Base: `esap_db`.
- Estrategia: **única base con esquemas por dominio**.

Esquemas identificados en inicialización/migraciones:

- `auth`
- `academic_registration`
- `certification`
- `control_interno`
- `internal_disciplinary_control`
- `legal_management`
- `requerimientos_oc`
- `audit`

7.2 Inicialización y migraciones

- DDL inicial en `db/init/00x_*`.
- Migraciones incrementales en `db/migrations/*.sql`.
- Scripts de despliegue ejecutan migraciones y registran en:
 - `auth.migrations_db_log`

7.3 Archivos y storage

Se usan carpetas locales/volumen para archivos:

- `backend/certification-service/uploads`
- `backend/internal-institutional-control-service/uploads`
- `backend/internal-disciplinary-control-service/uploads`
- `backend/legal-management-service/uploads`

Exposicion:

- Microservicios sirven `/uploads/*`.
- Gateway proxifica `{service}/uploads/*`.
- Nginx en entornos productivos publica `/services/certificados/uploads/*` por alias directo.

7.4 Redis

- Redis 7 en stack Docker.
- Servicios con variable `REDIS_URL` configurada en compose.
- Uso especifico depende de implementacion por servicio.

8. Infraestructura y despliegue

Diagrama de infraestructura disponible en el **Anexo Visual (14.1)**.

8.1 Docker Compose por entorno

Archivos:

- `docker-compose.yml` (local)
- `docker-compose.dev.yml`
- `docker-compose.qa.yml`
- `docker-compose.pre.yml`
- `docker-compose.prod.yml`

Stack base:

- frontend
- api-gateway
- 11 microservicios
- postgres
- redis

Stack extendido por entorno:

- `onlyoffice` esta en DEV, QA, PRE y PROD.

- `sonarqube` esta en DEV y QA.

8.2 Frontend containerizado

- Local: `Dockerfile.frontend` (dev server Vite).
- DEV/QA/PRE/PROD: Dockerfiles multi-stage (build Vite + Nginx runtime).
- `vite.config.ts` compila a `build/`.

8.3 Nginx de frontend

Funciones principales (`nginx.conf`):

- Servir SPA estaticamente.
- Proxy `/services/*` hacia `api-gateway:3000`.
- `client_max_body_size 250m` para uploads.
- Alias directo para uploads de certificados laborales.

8.4 Scripts de despliegue

Scripts:

- `deploy.dev.sh`
- `deploy.qa.sh`
- `deploy.pre.sh`
- `deploy.prod.sh`

Capacidades comunes:

- `up/down/restart`
- `rebuild` completo o por servicio
- `logs/status`
- backup de BD
- ejecucion de migraciones

9. Seguridad

Controles implementados:

- JWT en gateway y en servicios con guard propio.
- Lista de endpoints publicos controlada por patron.
- CORS configurable por entorno.
- Sanitizacion parcial de datos sensibles en auditoria.
- Encabezados de seguridad basicos en Nginx.

Observacion:

- El secreto JWT aparece con defaults en compose/documentacion de ejemplo; en produccion debe venir exclusivamente por secretos externos.

10. Integraciones externas

10.1 Microsoft OAuth (login)

- Frontend consume login Microsoft.
- `auth-service` implementa `POST /login/microsoft`.

10.2 Microsoft Graph (gestion legal)

- `legal-management-service` usa credenciales Azure para leer/sincronizar/enviar correo juridico.
- Variables: `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, `EMAIL_ACCOUNT_QA`.

10.3 OnlyOffice (control disciplinario)

- Frontend carga API JS de OnlyOffice.
- Backend disciplinario expone configuracion de editor y callback de guardado.

10.4 SMTP

- `notifications-service` envia correo simple, validacion y adjuntos.
- Otros servicios delegan notificaciones por llamada HTTP interna a `notifications-service`.

11. Observabilidad y pruebas

11.1 Observabilidad

- Auditoria transversal en gateway hacia `audit-service`.
- Logging por servicio (Nest logger + `console`).

11.2 Pruebas

Estado actual:

- Frontend: pruebas puntuales (`.spec/` `.test`) en pocos modulos.
- Backend: la mayoría de servicios conserva pruebas e2e scaffold base.

- Hay test de integracion en raiz con alto nivel de mock.

Conclusiones de calidad:

- Cobertura automatizada funcional real baja para un ecosistema de esta complejidad.
- Riesgo de regresiones alto en cambios transversales (gateway, auth, contratos API).

12. Brechas tecnicas y recomendaciones

12.1 Prioridad alta

1. Consolidar cliente API frontend

- Problema: coexistencia de `apiClient.ts` y `client.ts`, mas servicios legacy.
- Riesgo: inconsistencias de auth/errores/reintentos por modulo.
- Accion: unificar en un solo cliente y retirar wrappers duplicados.

1. Alinear contratos de endpoints declarados vs implementados

- Problema: `API_ENDPOINTS` contiene rutas no necesariamente soportadas hoy.
- Riesgo: deuda silenciosa y errores de integracion.
- Accion: generar contrato OpenAPI por servicio y validar en CI.

1. Reforzar testing de integracion real

- Problema: pocos tests ejecutando flujos reales entre gateway y servicios.
- Riesgo: regresiones en despliegues.
- Accion: suite minima critica (auth, certificados, disciplinario, legal, control interno).

12.2 Prioridad media

1. Resolver artefactos heredados de framework

- Problema: referencias `NEXT_PUBLIC_*` en frontend no alineadas con Vite.
- Accion: eliminar o migrar a `import.meta.env.VITE_*`.

1. Revisar coherencia de esquema en control institucional

- Problema: coexisten referencias `control_interno` e `internal_institutional_control` segun archivo/entorno.
- Accion: definir schema canonico y estandarizar compose, env, migraciones y codigo.

1. Externalizar completamente configuraciones sensibles

- Problema: defaults explicitos para secretos en compose de entorno.
- Accion: usar secretos del orquestador y validacion de arranque fail-fast.

12.3 Prioridad baja

1. Documentar mapa de ownership por modulo

- Accion: agregar responsables por servicio/modulo y SLA de cambios.

1. Homologar estilos de CORS y timeouts por servicio

- Accion: baseline comun para evitar comportamientos dispares entre microservicios.

13. Guia operativa rapida

13.1 Levantar QA

```
cp .env.example .env.qa  
./deploy.qa.sh up
```

13.2 Ver estado

```
./deploy.qa.sh status  
./deploy.qa.sh logs
```

13.3 Ejecutar migraciones

```
./deploy.qa.sh db-migrate
```

13.4 Rebuild puntual

```
./deploy.qa.sh rebuild-service auth-service
```

14. Anexo A - Referencias clave del codigo

Frontend:

- `src/main.tsx`
- `src/App.tsx`
- `src/components/esap/BackofficeApp.tsx`
- `src/config/environment.ts`
- `src/services/api/apiClient.ts`
- `src/services/api/client.ts`

Gateway:

- `backend/api-gateway/src/main.ts`
- `backend/api-gateway/src/app.module.ts`
- `backend/api-gateway/src/gateway/gateway.controller.ts`
- `backend/api-gateway/src/gateway/gateway.service.ts`
- `backend/api-gateway/src/gateway/proxy.config.ts`
- `backend/api-gateway/src/auth/jwt-auth.guard.ts`
- `backend/api-gateway/src/audit/audit.interceptor.ts`

Servicios de negocio:

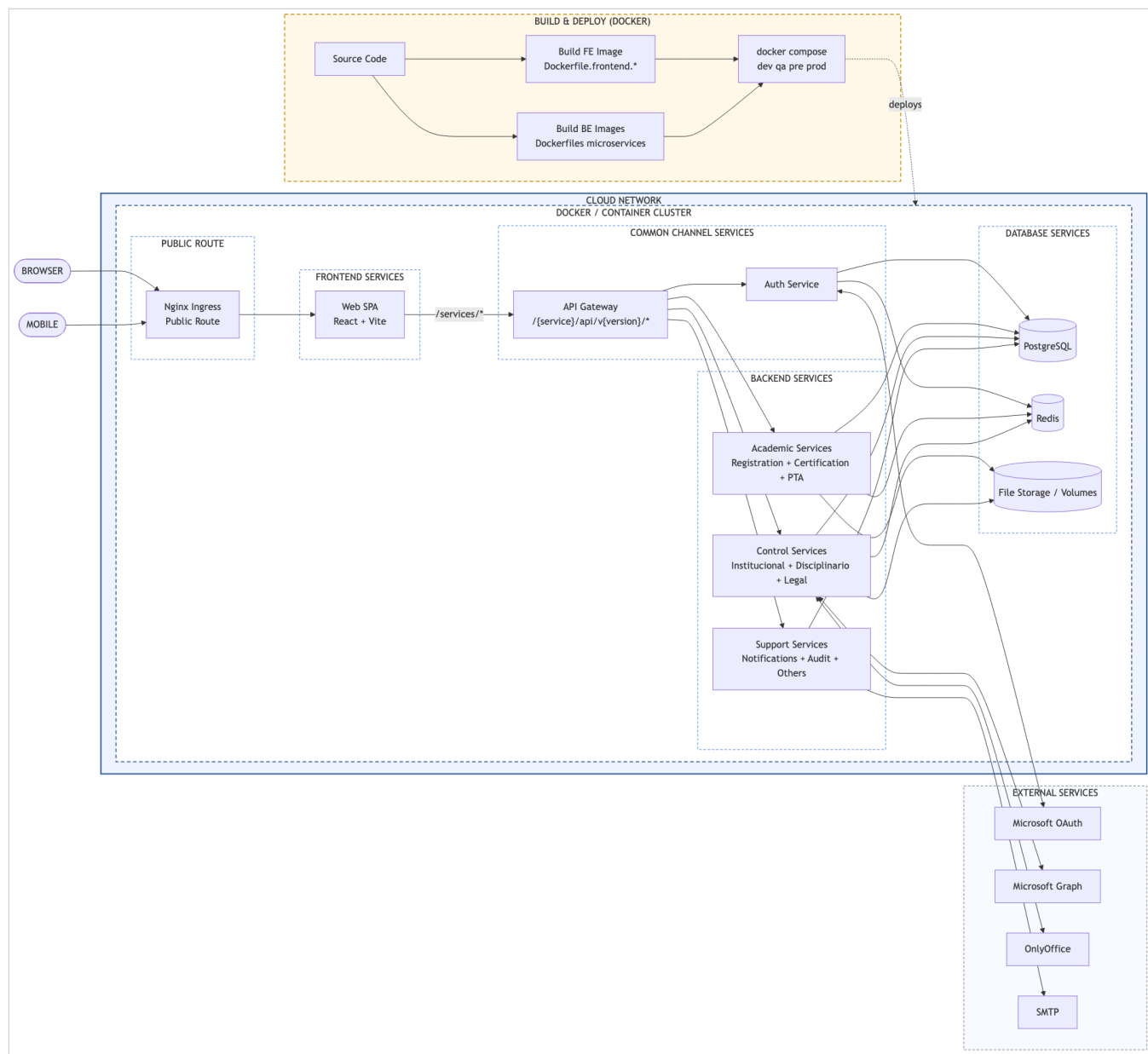
- `backend/auth-service/src/app.module.ts`
- `backend/academic-registration-service/src/graduation-certificates/graduation-certificates.controller.ts`
- `backend/certification-service/src/certificates/certificates.controller.ts`
- `backend/internal-disciplinary-control-service/src/app.module.ts`
- `backend/internal-institutional-control-service/src/app.module.ts`
- `backend/legal-management-service/src/app.module.ts`
- `backend/notifications-service/src/emails/emails.controller.ts`
- `backend/audit-service/src/audit/audit.controller.ts`

Infra y DB:

- `docker-compose.yml`
- `docker-compose.qa.yml`
- `Dockerfile.frontend*`
- `nginx.conf`
- `deploy.qa.sh`
- `db/init/*`
- `db/migrations/*`

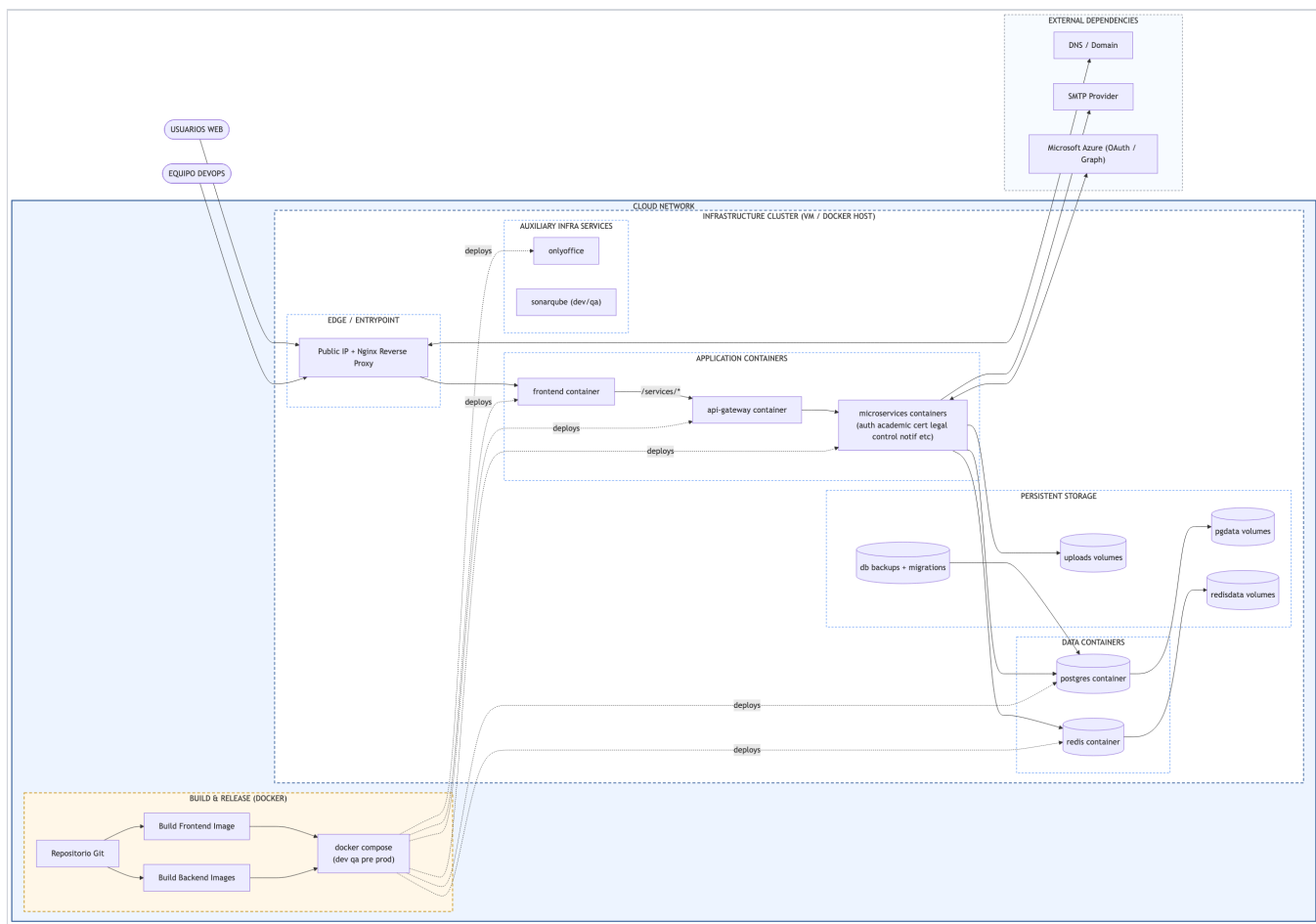
14.1 Anexo Visual - Diagramas e Imagenes

14.1.1 Diagrama de Arquitectura



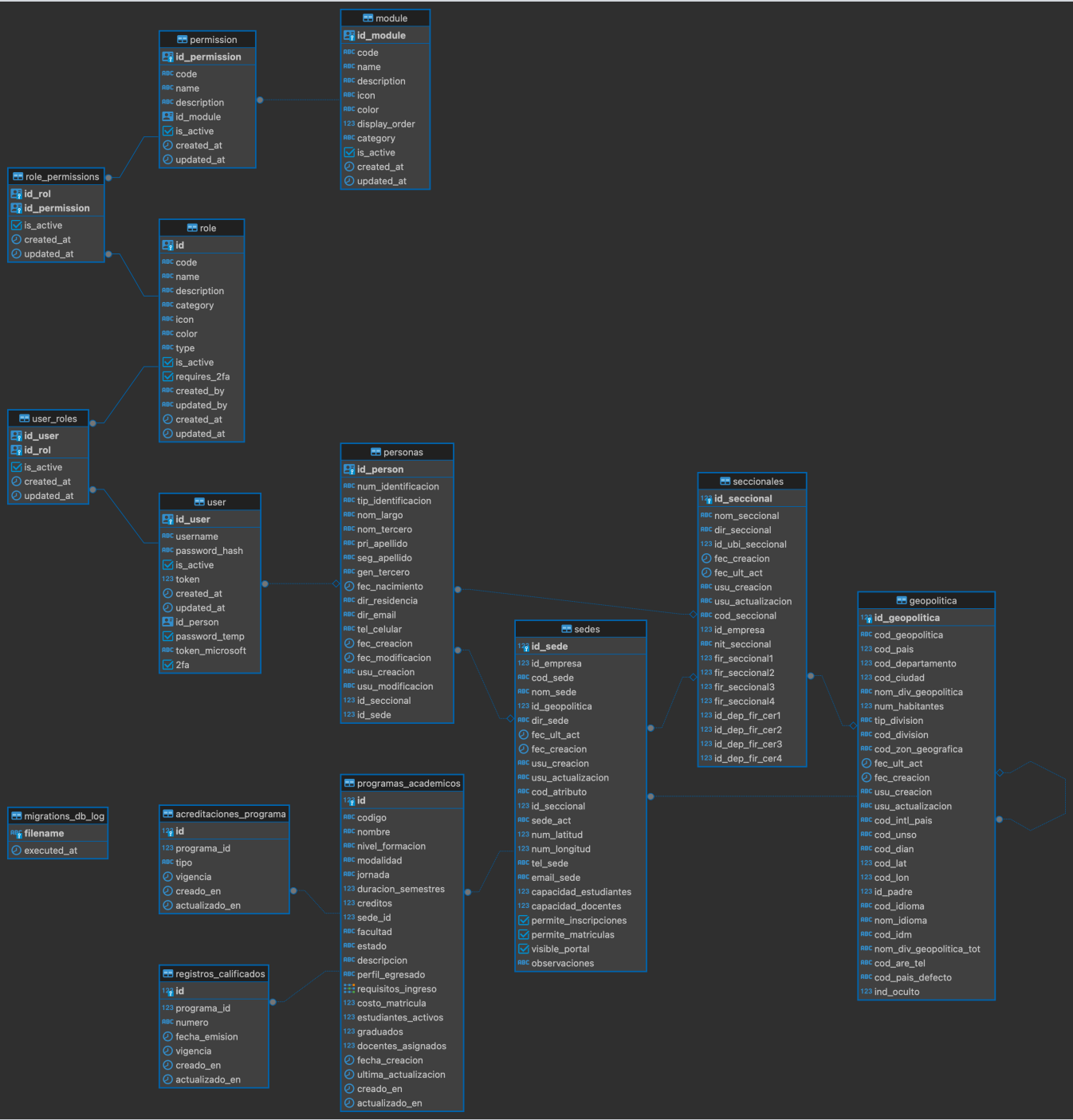
Fuente: `arquitectura_esap_cloud.mmd`

14.1.2 Diagrama de Infraestructura



Fuente: `infraestructura_esap.mmd`

14.1.3 Esquema BD Auth



15. Historial de cambios

Fecha	Version	Descripcion
2026-03-30	2026.03	Reemplazo de documentacion legacy por version as-is basada en codigo real